

MOTION DETECTION AND BOUNDING-BOX TRACKING SYSTEM

CSC 126 – Computer Vision Midterm Project Documentation

Project Type	Python desktop application using a webcam
Main Libraries	Python, OpenCV, NumPy
Computer Vision Topics	Image Operations; Image Segmentation; Motion Detection / Bounding Boxes
Prepared by	Jembo Magbanua, John Paul M. Toyco, Agustin Cordova

1. Introduction

Motion detection is a computer vision technique used to identify changes between video frames. This project is a Python desktop application that reads video from a webcam, compares the current frame with the previous frame, and highlights regions that appear to be moving. OpenCV is used to process the frames and draw visible bounding boxes around detected regions.

Description of the Problem or Use-Case

It can be difficult to notice movement continuously in a live camera feed. The system aims to provide a simple visual way to indicate movement by showing a motion status and drawing rectangles around regions that have changed between frames.

Objectives

- Capture frames from a webcam in real time.
- Apply image operations to prepare frames for comparison.
- Create a binary motion mask using frame difference and thresholding.
- Find changed regions and draw bounding boxes around sufficiently large areas.
- Demonstrate the use of at least three computer vision topics through a working application.

2. Techniques Used

- **Webcam frame capture (OpenCV):** Reads the current image from the webcam so the program can process the scene continuously.
- **Grayscale conversion — cv2.cvtColor:** Converts a color frame to shades of gray, making intensity comparisons simpler.
- **Gaussian blur — cv2.GaussianBlur:** Reduces small image noise that could otherwise create unwanted detections.
- **Frame difference — cv2.absdiff:** Compares the previous grayscale frame with the current frame to find pixel-level changes.
- **Thresholding — cv2.threshold:** Converts the difference image into a binary mask, marking sufficiently changed pixels as white and other pixels as black.
- **Dilation — cv2.dilate:** Expands white regions in the mask so nearby changed pixels can be grouped more easily.
- **Contour detection — cv2.findContours:** Finds connected regions in the binary motion mask.

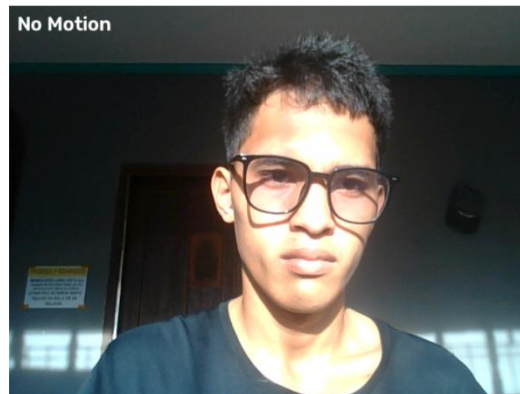
- **Contour-area filtering** — `cv2.contourArea`: Ignores very small regions that may be caused by noise.
- **Bounding rectangles** — `cv2.boundingRect` and `cv2.rectangle`: Calculates and draws rectangles around regions that pass the area filter.

3. Methodology and System Flow

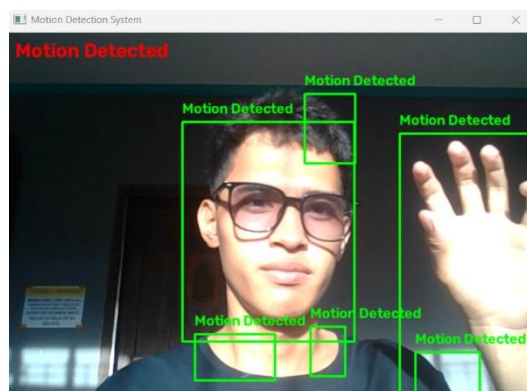
Step	Process	Purpose
1	Capture webcam frame	Read the current image from the camera.
2	Grayscale + Gaussian blur	Simplify the image and reduce noise.
3	Compare frames	Calculate pixel-level changes between current and previous frames.
4	Threshold + dilation	Create and improve the binary motion mask.
5	Find contours + filter area	Identify changed regions and ignore tiny regions.
6	Draw bounding boxes	Show detected motion visually on the frame.
7	Update previous frame	Use the current frame for the next comparison.

System flow: Webcam → Frame Capture → Grayscale → Gaussian Blur → Frame Difference → Threshold → Motion Mask → Contours → Area Filtering → Bounding Boxes → Motion Status.

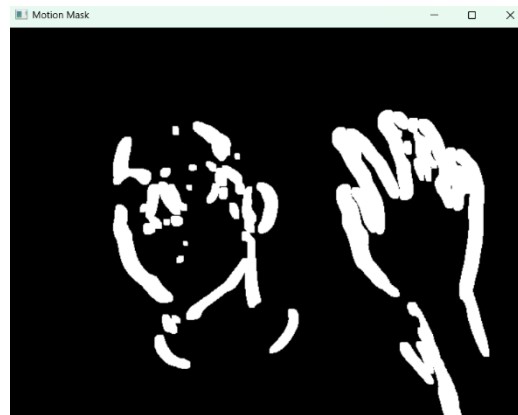
4. Testing and Results



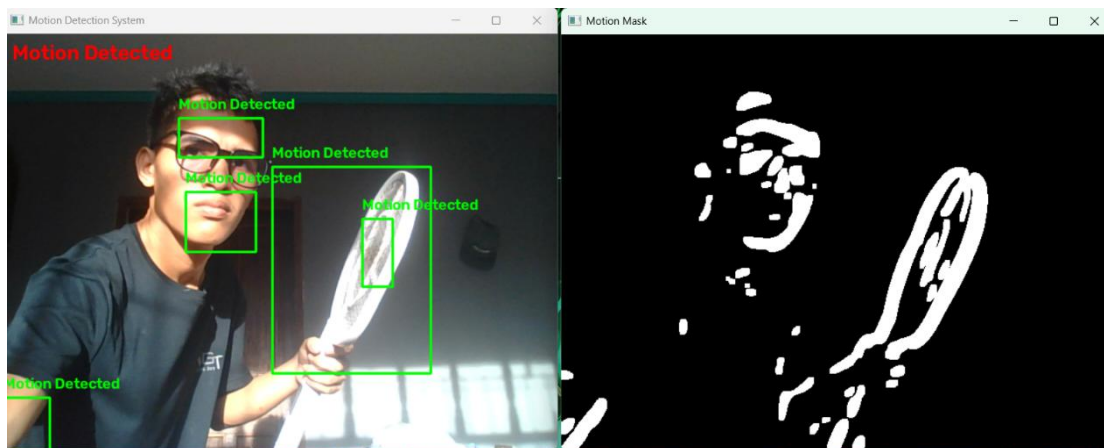
[Figure 1 No-motion state showing the webcam window and status.]



[Figure 2 Motion detected with a visible bounding box around a moving region]



[Figure 3 Motion Mask window showing white changed areas against a black background.]



[Figure 4 Multiple moving regions, if this can be demonstrated reliably.]

```
import cv2

CAMERA_INDEX = 0
MIN_AREA = 1000
THRESHOLD_VALUE = 25

# =====
# START CAMERA
# =====

cap = cv2.VideoCapture(CAMERA_INDEX)

if not cap.isOpened():
    print("Error: Could not open the webcam.")
    print("Try changing CAMERA_INDEX from 0 to 1.")
    exit()

print("Motion Detection System Started")
print("Move something in front of the camera.")
print("Press Q to quit.")

# =====
# PREVIOUS FRAME
# =====

previous_gray = None
```

[Figure 5 VS Code or editor screenshot showing the key OpenCV functions.]

5. Discussion of Accuracy, Behavior, and Limitations

The system detects movement by comparing consecutive frames, rather than by recognizing objects. It is expected to highlight regions when visible changes are large enough to pass the threshold and minimum-area filter. The result depends on the camera view, lighting, threshold settings, and how much the scene changes.

Expected Behavior to Verify During Testing

Test	Procedure	Expected behavior
No motion	Keep the scene still.	The status should indicate no motion, and large boxes should generally not appear.
Hand movement	Move a hand in front of the webcam.	A motion status and a box should appear around the changed region.
Moving object	Move an object within the camera view.	The changed region should be highlighted if it passes the detection settings.
Several changes	Move more than one object or region.	One or more boxes may appear.
Lighting change	Change the lighting or create a shadow.	The system may incorrectly detect this as motion.

Limitations

- Camera movement can make much of the scene appear to be moving.
- Changes in lighting, shadows, or screen flicker may cause false detections.
- Small or slow movements may not be detected depending on the threshold and minimum-area settings.
- The program detects visual changes; it does not understand what the moving object is.
- Bounding boxes may change between frames and do not represent persistent object identity.

6. Conclusion and Future Improvements

The Motion Detection and Bounding-Box Tracking System demonstrates a basic real-time motion detection workflow using Python, OpenCV, NumPy, and a webcam. It combines grayscale conversion, Gaussian blur, frame differencing, thresholding, dilation, contour analysis, and bounding box drawing to highlight regions that change between consecutive frames. This project provides a practical introduction to image operations, image segmentation, and motion detection. Its results are affected by lighting, camera stability, and detection settings, and it does not perform object recognition or persistent identity tracking.

Possible future improvements include recording detected motion, adding alerts, using background subtraction for more stable detection, applying a trained object detector to recognize people, and adding persistent IDs with an object-tracking algorithm.